

TD Nr 12 – La clause FROM

La clause FROM est souvent perçue comme la plus simple d'une requête SQL, mais c'est en réalité le **moteur de votre requête**. C'est ici que vous définissez la source des données et que l'optimiseur de la base de données commence son travail.

1. L'ordre d'exécution logique

Contrairement à l'ordre d'écriture (où l'on commence par SELECT), SQL traite la clause FROM en **premier**.

- **Pourquoi c'est important ?** Tant que le FROM n'est pas défini, la base de données ne sait pas quelles colonnes sont disponibles pour le SELECT ou le WHERE.

2. Les sources multiples (Joins)

La clause FROM ne se limite pas à une seule table. Elle permet de combiner plusieurs sources de données via des jointures.

```
SELECT clients.nom, commandes.date_achat
FROM clients
INNER JOIN commandes ON clients.id = commandes.client_id;
```

La clause JOIN est ce qui donne tout son sens au "R" de SQL (Relationnel). Elle permet de lier plusieurs tables entre elles dans une seule clause SELECT, à condition qu'elles partagent un point commun (généralement une clé étrangère).

Voici comment orchestrer la fusion de vos données.

a) La syntaxe de base

Pour faire un JOIN, on utilise généralement le mot-clé ON pour définir la condition de liaison (quelle colonne de la table A correspond à quelle colonne de la table B).

```
SELECT Employes.nom, Departements.nom_dept
FROM Employes
JOIN Departements ON Employes.id_dept = Departements.id;
```

b) Les différents types de JOIN

Il existe plusieurs façons de lier les tables selon que vous vouliez garder les données qui n'ont pas de correspondance ou non.

INNER JOIN (Le plus courant)

Il ne retourne que les lignes qui ont une correspondance dans les deux tables.

- *Exemple* : Si un employé n'est rattaché à aucun département, il n'apparaîtra pas dans la liste.

LEFT JOIN (ou LEFT OUTER JOIN)

Il retourne toutes les lignes de la table de gauche, même s'il n'y a pas de correspondance à droite.

- *Exemple* : Utile pour lister tous les employés, même ceux qui n'ont pas encore de département (la colonne département affichera NULL).

RIGHT JOIN

L'inverse du Left Join : il garde toutes les lignes de la table de droite.

c) Exemple concret avec plusieurs tables

On peut enchaîner les jointures pour récupérer des informations éparpillées.

```
SELECT
    C.nom_client,
    CO.date_commande,
    P.nom_produit
FROM Clients AS C
JOIN Commandes AS CO ON C.id = CO.client_id
JOIN Produits AS P ON CO.produit_id = P.id;
```

Note : L'utilisation d'alias (AS C, AS CO) est ici indispensable pour la lisibilité.

d) Les bonnes pratiques

- Toujours préfixer les colonnes : Si les deux tables ont une colonne id, écrivez Employees.id ou E.id pour éviter que SQL ne soit "ambigu".
- Utiliser des alias courts : Cela rend la requête beaucoup plus facile à lire.
- Vérifier les index : Les jointures sont gourmandes en ressources. Assurez-vous que les colonnes utilisées dans le ON sont indexées dans la base de données.

INNER	Uniquement les "couples" parfaits.
LEFT	Tout à gauche + ce qui correspond à droite.
FULL	Tout, partout (si supporté par le SGBD).